

Complete NLP Cheat Sheet

Table of Contents

1. [NLP Fundamentals](#)
 2. [Text Preprocessing](#)
 3. [Text Representation](#)
 4. [Classical NLP Techniques](#)
 5. [Deep Learning for NLP](#)
 6. [Transformer Architecture](#)
 7. [BERT Model](#)
 8. [T5 Model](#)
 9. [GPT Models](#)
 10. [Advanced Architectures](#)
 11. [Common NLP Tasks](#)
 12. [Evaluation Metrics](#)
 13. [Prompt Engineering](#)
 14. [Fine-tuning Strategies](#)
 15. [NLP Challenges](#)
 16. [Tools and Libraries](#)
-

NLP Fundamentals

Natural Language Processing (NLP) is the field of AI focused on enabling computers to understand, interpret, and generate human language.

Core Concepts

- **Corpus:** A large collection of text documents
- **Token:** Individual unit of text (word, subword, character)
- **Vocabulary:** Set of all unique tokens in a corpus
- **Embeddings:** Dense vector representations of text
- **Context:** Surrounding words that give meaning to a token
- **Semantic Similarity:** Measure of meaning similarity between texts
- **Syntactic Analysis:** Understanding grammatical structure
- **Pragmatics:** Understanding intended meaning in context

NLP Pipeline Stages

1. **Data Collection:** Gathering text data from various sources
2. **Text Preprocessing:** Cleaning and normalizing text
3. **Feature Extraction:** Converting text to numerical representations
4. **Model Training:** Learning patterns from data
5. **Evaluation:** Measuring model performance
6. **Deployment:** Putting model into production
7. **Monitoring:** Tracking performance over time

Types of NLP

Rule-Based NLP:

- Uses hand-crafted linguistic rules
- Regular expressions, grammar rules
- Deterministic, interpretable
- Limited scalability

Statistical NLP:

- Uses probabilistic models
- N-grams, Hidden Markov Models
- Learns from data
- Better generalization

Neural NLP:

- Uses deep learning models
- Learns complex patterns
- State-of-the-art performance
- Requires large datasets

Hybrid Approaches:

- Combines multiple methods
 - Leverages strengths of each
 - Common in production systems
-

Text Preprocessing

Tokenization

Breaking text into smaller units (tokens).

Types:

- **Word Tokenization:** "Hello world" → ["Hello", "world"]
- **Subword Tokenization:** "unhappiness" → ["un", "happiness"]
- **Character Tokenization:** "cat" → ["c", "a", "t"]
- **Sentence Tokenization:** Splitting text into sentences

Common Preprocessing Steps

1. **Lowercasing:** Convert all text to lowercase
2. **Removing punctuation:** Strip special characters
3. **Removing stop words:** Filter common words (the, is, at)
4. **Stemming:** Reduce words to root form (running → run)
5. **Lemmatization:** Reduce to dictionary form (better → good)
6. **Removing numbers/URLs/emails:** Clean irrelevant data
7. **Handling contractions:** can't → cannot
8. **Unicode normalization:** Handle different character encodings
9. **Spell correction:** Fix typos and misspellings
10. **HTML/XML tag removal:** Clean web-scraped data

Advanced Tokenization Methods

- **Byte Pair Encoding (BPE):** Merges frequent character pairs iteratively
- **WordPiece:** Used by BERT, similar to BPE
- **SentencePiece:** Language-independent tokenization
- **Unigram:** Probabilistic tokenization method
- **BPE-Dropout:** Adds randomness for robustness

Text Normalization Techniques

Case Folding:

- **Lowercase:** Standard approach
- **Truecasing:** Preserve proper nouns
- **Mixed:** Task-dependent

Noise Removal:

- HTML tags and entities
- Special characters
- Repeated characters (sooo → so)
- Emojis (keep or remove based on task)

Standardization:

- Date formats
 - Number formats
 - Abbreviations expansion
 - Slang normalization
-

Text Representation

Traditional Methods

1. One-Hot Encoding

- Binary vector with 1 at token's index
- Size = vocabulary size
- No semantic information
- Sparse representation
- Example: "cat" = [0,0,1,0,0,...,0]

2. Bag of Words (BoW)

- Count frequency of each word
- Ignores word order and grammar
- Sparse representation
- Simple but effective baseline

3. TF-IDF (Term Frequency-Inverse Document Frequency)

- Weights words by importance
- $TF = (\text{word count in document}) / (\text{total words in document})$
- $IDF = \log(\text{total documents} / \text{documents containing word})$
- $TF-IDF = TF \times IDF$

- Reduces weight of common words
- Increases weight of rare, distinctive words

4. N-gram Features

- Captures local word order
- Bigrams, trigrams, etc.
- Better than BoW for some tasks
- Vocabulary size grows exponentially

Word Embeddings

Word2Vec (2013)

Two architectures:

- **CBOW (Continuous Bag of Words)**: Predict word from context
- **Skip-gram**: Predict context from word

Training Methods:

- Negative sampling
- Hierarchical softmax

Properties:

- Creates dense vectors (100-300 dimensions)
- Captures semantic meaning
- Linear relationships: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$
- Similar words have similar vectors
- Trained on large corpora (Google News, Wikipedia)

GloVe (Global Vectors, 2014)

- Based on word co-occurrence statistics
- Combines global matrix factorization with local context
- Pre-trained vectors available (6B, 42B, 840B tokens)
- Captures both semantic and syntactic relationships

FastText (2016)

- Extension of Word2Vec
- Represents words as bag of character n-grams

- Handles out-of-vocabulary words better
- Good for morphologically rich languages
- Can generate embeddings for unseen words

Contextual Embeddings

ELMo (Embeddings from Language Models, 2018)

- Bidirectional LSTM-based
- Character-level inputs
- Word representations vary by context
- Pre-trained on large corpus
- Can be used as features in downstream tasks

CoVe (Contextualized Word Vectors, 2017)

- Trained using machine translation
- Transfer learning from MT to other tasks

BERT, GPT, and Transformer-based:

- Deep contextual representations
- Pre-trained on massive datasets
- Fine-tunable for specific tasks
- State-of-the-art across many benchmarks

Sentence and Document Embeddings

Doc2Vec (Paragraph Vectors)

- Extension of Word2Vec to documents
- Two variants: PV-DM and PV-DBOW
- Fixed-length vector for variable-length text

Sentence-BERT (SBERT)

- Uses Siamese/triplet networks
- Efficient semantic similarity computation
- Good for clustering and retrieval

Universal Sentence Encoder

- Trained on multiple tasks

- Transformer or DAN architecture
- Available via TensorFlow Hub

InferSent

- Trained on natural language inference
 - Captures sentence semantics
-

Classical NLP Techniques

N-grams and Language Models

Sequence of N consecutive tokens.

- **Unigram:** Single word ("dog")
- **Bigram:** Two words ("happy birthday")
- **Trigram:** Three words ("New York City")
- **4-gram, 5-gram:** Higher order n-grams

Language Models:

- Assign probability to sequences
- $P(\text{sentence}) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \dots$
- Markov assumption for simplification
- Used in speech recognition, MT, text generation

Smoothing Techniques:

- Laplace (add-one) smoothing
- Good-Turing smoothing
- Kneser-Ney smoothing
- Handle unseen n-grams

Part-of-Speech (POS) Tagging

Labeling words with grammatical categories.

Common Tags:

- NN (noun), NNS (plural noun)
- VB (verb), VBD (past tense verb)

- JJ (adjective), RB (adverb)
- DT (determiner), IN (preposition)

Methods:

- Rule-based taggers
- HMM (Hidden Markov Model) taggers
- Maximum Entropy Markov Models (MEMM)
- CRF (Conditional Random Fields)
- Neural taggers (BiLSTM-CRF)

Named Entity Recognition (NER)

Identifying and classifying named entities.

Common Entities:

- PERSON: Names of people
- ORGANIZATION: Companies, agencies
- LOCATION: Cities, countries, landmarks
- DATE: Dates and times
- MONEY: Monetary values
- PERCENT: Percentages
- PRODUCT: Products and services
- EVENT: Named events

Approaches:

- Rule-based (gazetteers, patterns)
- Statistical (CRF, HMM)
- Neural (BiLSTM-CRF, BERT-based)

Annotation Schemes:

- IOB (Inside, Outside, Beginning)
- IOBES (IOB + End, Single)

Syntactic Parsing

Constituency Parsing:

- Breaks sentence into nested constituents

- Creates parse trees with phrase structure
- NP (Noun Phrase), VP (Verb Phrase), etc.

Dependency Parsing:

- Shows grammatical relationships between words
- Creates directed graph
- Relations: nsubj (subject), dobj (object), amod (modifier)

Methods:

- Probabilistic Context-Free Grammars (PCFG)
- Transition-based parsing
- Graph-based parsing
- Neural parsers (BiLSTM, Transformers)

Semantic Analysis

Word Sense Disambiguation (WSD):

- Determine correct meaning of polysemous words
- "bank" → financial institution vs. river bank
- Knowledge-based or supervised learning

Semantic Role Labeling (SRL):

- Identifies predicate-argument structure
- Who did what to whom, when, where, why
- Frame-based representations

Coreference Resolution:

- Identify when different expressions refer to same entity
- "John went to store. He bought milk." (He = John)
- Mention detection + mention clustering

Topic Modeling

Latent Dirichlet Allocation (LDA):

- Probabilistic generative model
- Documents as mixtures of topics
- Topics as distributions over words

- Unsupervised learning

Non-negative Matrix Factorization (NMF):

- Matrix factorization approach
- Document-term matrix $\rightarrow$ document-topic $\times$ topic-term
- Interpretable topics

Latent Semantic Analysis (LSA):

- SVD on document-term matrix
 - Dimensionality reduction
 - Captures latent semantic structure
-

Deep Learning for NLP

Recurrent Neural Networks (RNN)

- Process sequential data one token at a time
- Maintain hidden state to capture context
- Share parameters across time steps
- **Problem:** Vanishing/exploding gradients
- Difficulty learning long-term dependencies

Equations:

- $h_t = \tanh(W_{hh} \times h_{t-1} + W_{xh} \times x_t + b_h)$
- $y_t = W_{hy} \times h_t + b_y$

Long Short-Term Memory (LSTM)

- Special RNN architecture with gates
- **Forget gate:** Decides what to discard from cell state
- **Input gate:** Decides what new information to add
- **Output gate:** Decides what to output
- Better at capturing long-term dependencies
- Solves vanishing gradient problem

Bidirectional LSTM:

- Processes sequence in both directions
- Forward LSTM + Backward LSTM
- Combines information from past and future
- Better context understanding

Gated Recurrent Unit (GRU)

- Simpler variant of LSTM
- Two gates: reset and update
- Fewer parameters than LSTM
- Faster training
- Often similar performance to LSTM

Convolutional Neural Networks (CNN) for Text

Architecture:

- 1D convolutions over text
- Multiple filter sizes (3, 4, 5 grams)
- Max pooling to extract features
- Fully connected layers for classification

Advantages:

- Captures local patterns
- Parallel computation (faster than RNNs)
- Good for classification tasks
- Effective feature extraction

Applications:

- Sentence classification
- Text categorization
- Relation extraction

Attention Mechanism

Key Concepts:

- Allows model to focus on relevant parts of input
- Computes weighted sum of input representations

- Weights learned during training
- Dynamic context vectors

Types:

1. Additive Attention (Bahdanau):

- $\text{score}(h_t, s_i) = v^T \tanh(W_1 h_t + W_2 s_i)$

2. Multiplicative Attention (Luong):

- $\text{score}(h_t, s_i) = h_t^T W s_i$

3. Scaled Dot-Product Attention:

- $\text{score}(Q, K) = (Q \times K^T) / \sqrt{d_k}$
- Used in Transformers

Self-Attention:

- Attention over same sequence
- Each element attends to all elements
- Captures dependencies within sequence

Sequence-to-Sequence Models

Encoder-Decoder Architecture:

- Encoder: Processes input sequence $\rightarrow$ context vector
- Decoder: Generates output sequence from context
- Used for translation, summarization, etc.

With Attention:

- Decoder attends to encoder hidden states
 - Different parts of input for each output
 - Solves bottleneck problem
 - Significantly improves performance
-

Transformer Architecture

Overview

The Transformer (2017, "Attention Is All You Need") revolutionized NLP by relying entirely on attention mechanisms, eliminating recurrence.

Key Components

1. Self-Attention Mechanism

Allows each token to attend to all other tokens in the sequence.

Process:

1. Create Query (Q), Key (K), Value (V) matrices from input embeddings
2. Compute attention scores: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$
3. Each token gets context-aware representation

Intuition:

- Query: What am I looking for?
- Key: What do I contain?
- Value: What do I actually communicate?

Why scaling by $\sqrt{d_k}$?

- Prevents dot products from becoming too large
- Keeps gradients stable
- Improves training dynamics

Why it works:

- Captures long-range dependencies efficiently
- Parallel computation (unlike RNNs)
- Direct connections between all positions
- No gradient vanishing across long distances

2. Multi-Head Attention

Runs multiple self-attention operations in parallel.

Benefits:

- Capture different types of relationships

- Focus on different representation subspaces
- Different heads learn different patterns
- Richer representations

Process:

1. Project Q, K, V into h different subspaces
2. Apply attention in each subspace
3. Concatenate results
4. Project back to original dimension

Formula: $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$ where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

3. Positional Encoding

Since Transformers process all tokens simultaneously, they need explicit position information.

Sinusoidal Positional Encoding:

- $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$
- $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$
- Deterministic, no parameters
- Can extrapolate to longer sequences

Learned Positional Embeddings:

- Trained parameters
- Fixed maximum sequence length
- Often used in practice

Relative Position Encodings:

- Encode relative distances between tokens
- More flexible
- Better generalization

4. Feed-Forward Networks

Applied to each position independently and identically.

Architecture:

- Two linear transformations with activation

- $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$
- Usually: $d_{\text{model}} \rightarrow 4 \times d_{\text{model}} \rightarrow d_{\text{model}}$
- ReLU or GELU activation

Purpose:

- Add non-linearity
- Process each position's representation
- Increase model capacity

5. Layer Normalization & Residual Connections

Layer Normalization:

- Normalize across features for each example
- $\text{LayerNorm}(x) = \gamma(x - \mu) / \sigma + \beta$
- Stabilizes training
- Reduces internal covariate shift

Residual Connections:

- Add input to output: $x + \text{Sublayer}(x)$
- Enables gradient flow
- Allows training very deep networks
- Identity mapping when needed

Combined (Add & Norm):

- $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$
- Standard in Transformers
- Two variants: Pre-LN and Post-LN

Encoder-Decoder Architecture

Encoder:

- Stack of N identical layers (typically 6-12)
- Each layer has two sub-layers:
 1. Multi-head self-attention
 2. Position-wise feed-forward network
- Residual connections and layer normalization
- Processes entire input in parallel

- Outputs contextualized representations

Decoder:

- Stack of N identical layers
- Each layer has three sub-layers:
 1. Masked multi-head self-attention
 2. Multi-head attention over encoder output
 3. Position-wise feed-forward network
- Masked attention prevents looking at future positions
- Autoregressive generation
- Cross-attention connects to encoder

Information Flow:

1. Input → Encoder → Context representations
2. Previous outputs → Decoder self-attention
3. Decoder attends to encoder outputs
4. Generate next token
5. Repeat autoregressively

Transformer Variants by Architecture

Encoder-only (Autoencoding):

- BERT, RoBERTa, ALBERT, DeBERTa
- Bidirectional context
- Best for understanding tasks
- Classification, NER, QA (extractive)

Decoder-only (Autoregressive):

- GPT series, LLaMA, Mistral, Falcon
- Unidirectional (left-to-right)
- Best for generation tasks
- Text completion, creative writing, code

Encoder-Decoder (Sequence-to-Sequence):

- T5, BART, mBART, mT5
- Both bidirectional and autoregressive

- Best for transformation tasks
- Translation, summarization, QA (generative)

Advanced Transformer Variants

Longformer:

- Efficient attention for long documents
- Local + global attention
- Linear complexity in sequence length

BigBird:

- Sparse attention patterns
- Random, window, and global attention
- Handles sequences up to 4096 tokens

Reformer:

- Locality-sensitive hashing for attention
- Reversible layers
- Reduces memory requirements

Linformer:

- Low-rank approximation of attention
- Linear complexity
- Maintains performance

Performers:

- Fast attention via orthogonal random features
 - Linear time and space complexity
 - FAVOR+ algorithm
-

BERT Model

BERT: Bidirectional Encoder Representations from Transformers (2018)

Architecture:

- Encoder-only Transformer
- BERT-Base: 12 layers, 768 hidden size, 12 attention heads, 110M parameters
- BERT-Large: 24 layers, 1024 hidden size, 16 attention heads, 340M parameters
- Max sequence length: 512 tokens

Key Innovation: Bidirectional Context

Unlike previous models (like GPT) that read text left-to-right or right-to-left, BERT reads in both directions simultaneously, creating deeper understanding of context.

Comparison:

- GPT: Unidirectional (left-to-right)
- ELMo: Shallow bidirectional (concatenation)
- BERT: Deep bidirectional (joint conditioning)

Pre-training Tasks

1. Masked Language Modeling (MLM)

Process:

- Randomly mask 15% of tokens in input
- Predict masked tokens using bidirectional context
- Example: "The [MASK] sat on the mat" → predict "cat"
- Forces model to understand context from both directions

Masking Strategy:

- 80%: Replace with [MASK] token
- 10%: Replace with random word
- 10%: Keep unchanged

Why mixed strategy?

- [MASK] token doesn't appear during fine-tuning
- Random words force model to rely on context
- Unchanged words reduce mismatch

2. Next Sentence Prediction (NSP)

Process:

- Given sentence pairs (A, B)

- Predict if B actually follows A (IsNext vs NotNext)
- 50% positive pairs, 50% random pairs

Purpose:

- Understand sentence relationships
- Useful for QA and NLI tasks
- Later shown to be less important (RoBERTa removed it)

Special Tokens

- **[CLS]**: Classification token (start of sequence)
 - Final state used for classification
 - Aggregates sequence information
- **[SEP]**: Separator between sentences
- **[MASK]**: Masked token placeholder
- **[PAD]**: Padding for batch processing

Input Representation

BERT combines three types of embeddings:

1. **Token Embeddings**: WordPiece vocabulary (30,000 tokens)
2. **Segment Embeddings**: Sentence A vs Sentence B
3. **Position Embeddings**: Absolute position (0-511)

Total Input = Token Embedding + Segment Embedding + Position Embedding

Pre-training Details

Data:

- BooksCorpus (800M words)
- English Wikipedia (2,500M words)
- Total: 3.3 billion words

Training:

- 1 million steps
- Batch size: 256 sequences
- Sequence length: 512 tokens
- Learning rate: 1e-4

- Warmup: 10,000 steps
- Hardware: 4-16 Cloud TPUs

Fine-tuning BERT

BERT uses a two-stage process:

1. **Pre-training:** Learn general language understanding
2. **Fine-tuning:** Adapt to specific task

Single Sentence Classification:

- Input: [CLS] sentence [SEP]
- Use [CLS] representation
- Add classification layer
- Tasks: Sentiment analysis, spam detection

Sentence Pair Classification:

- Input: [CLS] sentence A [SEP] sentence B [SEP]
- Use [CLS] representation
- Tasks: Natural Language Inference, paraphrase detection

Question Answering (SQuAD):

- Input: [CLS] question [SEP] passage [SEP]
- Predict start and end positions of answer
- Two output vectors: start scores, end scores
- Answer = span from start to end

Named Entity Recognition:

- Token-level classification
- Each token gets a label
- Labels: B-PER, I-PER, B-ORG, I-ORG, O, etc.
- Use representation of each token

Fine-tuning Strategy:

- Usually 2-4 epochs
- Small learning rate (2e-5, 3e-5, 5e-5)
- Batch size: 16 or 32

- Fast convergence

BERT Variants and Improvements

RoBERTa (Robustly Optimized BERT, 2019):

- Removed NSP task
- Dynamic masking (different each epoch)
- Larger batches and learning rate
- More training data (160GB text)
- Trained longer
- Consistently better than BERT

ALBERT (A Lite BERT, 2019):

- Factorized embedding parameterization
- Cross-layer parameter sharing
- Sentence-order prediction (SOP) instead of NSP
- 18x fewer parameters than BERT-large
- Similar or better performance

DistilBERT (2019):

- Knowledge distillation from BERT
- 40% fewer parameters
- 60% faster
- Retains 97% of BERT's performance
- Good for resource-constrained environments

ELECTRA (2020):

- Replaced token detection instead of MLM
- Discriminator vs generator setup
- More sample-efficient
- Better performance with less compute
- Particularly good for smaller models

DeBERTa (Decoding-enhanced BERT, 2020):

- Disentangled attention mechanism
- Enhanced mask decoder

- Virtual adversarial training
- State-of-the-art on SuperGLUE

SpanBERT (2019):

- Masks contiguous random spans
- Span boundary objective
- Better for span-based tasks
- Improved on QA and coreference

ERNIE (Enhanced Representation through Knowledge):

- Masks entities and phrases
- Incorporates knowledge graphs
- Better world knowledge

Multilingual BERT

mBERT (Multilingual BERT):

- Trained on 104 languages
- Shared vocabulary (110k WordPieces)
- Zero-shot cross-lingual transfer
- Not explicitly aligned across languages

XLNet-RoBERTa:

- Trained on 100 languages
- 2.5TB CommonCrawl data
- Better cross-lingual performance
- No language embeddings needed

Advantages of BERT

- Deep bidirectional context understanding
- Transfer learning: pre-train once, fine-tune many times
- State-of-the-art on 11 NLP tasks at release
- Handles polysemy (context-dependent meanings)
- Relatively easy to fine-tune
- Strong zero-shot and few-shot capabilities

Limitations

- Computationally expensive (large models)
- Fixed maximum sequence length (512 tokens)
- Cannot generate text fluently (encoder-only)
- Pre-training is very resource-intensive
- Static vocabulary (can't handle new words well)
- Independence assumption in MLM

Applications

- Search engines (Google Search uses BERT)
 - Question answering systems
 - Sentiment analysis
 - Text classification
 - Named entity recognition
 - Information extraction
 - Document ranking
 - Chatbots and dialogue systems
-

T5 Model

T5: Text-to-Text Transfer Transformer (2019)

Core Philosophy: Every NLP task is framed as a text-to-text problem.

Key Insight: Unified framework allows using the same model, loss function, and hyperparameters for all tasks.

Unified Text-to-Text Framework

T5 converts all tasks to the same format:

- **Input:** Text prompt describing the task + input
- **Output:** Text response

Examples:

- **Translation:** "translate English to German: Hello" → "Hallo"
- **Summarization:** "summarize: [article text]" → "[summary]"
- **Classification:** "sst2 sentence: I love this movie!" → "positive"

- **Question Answering:** "question: What is NLP? context: [text]" → "[answer]"
- **Regression:** "stsb sentence1: X sentence2: Y" → "4.5"
- **Coreference:** "wsc: [text with pronouns]" → "True" or "False"

Architecture

Encoder-Decoder Transformer:

- Based on original Transformer architecture
- Relative position embeddings instead of absolute
- Simplified layer normalization (no bias, no subtraction)
- Encoder and decoder have same structure

Model Sizes:

- **T5-Small:** 60M parameters (8 layers, 512 hidden)
- **T5-Base:** 220M parameters (12 layers, 768 hidden)
- **T5-Large:** 770M parameters (24 layers, 1024 hidden)
- **T5-3B:** 3 billion parameters
- **T5-11B:** 11 billion parameters

Scaling: Performance improves consistently with model size.

Pre-training Approach

Unsupervised Objective: Span Corruption

Process:

1. Corrupt random spans of text (avg. length 3 tokens)
2. Replace each span with unique sentinel token
3. Model predicts all corrupted spans

Example:

- Original: "Thank you for inviting me to your party last week"
- Corrupted: "Thank you <X> to <Y> last week"
- Target: "<X> for inviting me <Y> your party <Z>"

Why span corruption?

- More challenging than single token prediction
- Forces understanding of longer context

- Better than random token replacement
- Balances between reconstruction and generation

C4 Dataset (Colossal Clean Crawled Corpus):

- 750 GB of clean English web text
- Filtered from Common Crawl
- Removed duplicates, incomplete sentences
- Cleaned markup and offensive content

Multi-Task Training

Supervised Multi-Task Learning:

- Trains on diverse tasks simultaneously
- Tasks share parameters
- Improves generalization

Task Mixture:

- Translation (WMT datasets)
- Summarization (CNN/Daily Mail, XSum)
- Question Answering (SQuAD, Natural Questions)
- Classification (GLUE tasks)
- Common sense reasoning (COPA, WSC)

Mixing Strategy:

- Examples-proportional mixing
- Temperature-based sampling
- Equal mixing across tasks
- Best: proportional with temperature

Key Innovations

1. Comprehensive Ablation Study

T5 paper systematically compared:

- Architectures (encoder-decoder vs decoder-only vs prefix-LM)
- Pre-training objectives (MLM, prefix-LM, deshuffling)
- Datasets (C4, Wikipedia, WebText)

- Transfer approaches (fine-tuning, adapter layers)
- Scaling (model size, data size, training steps)

Findings:

- Encoder-decoder works best for most tasks
- Denoising objectives (like span corruption) are effective
- Scaling model and data improves performance
- Pre-training on in-domain data helps

2. Task Prefixes for Multi-Task Learning

Examples:

- "translate English to French:"
- "summarize:"
- "cola sentence:"
- "stsb sentence1: ... sentence2: ..."

Benefits:

- Explicit task identification
- No task-specific parameters needed
- Easy to add new tasks
- Supports zero-shot learning

3. Transfer Learning Framework

Pre-training:

- Unsupervised learning on massive corpus
- Learn general language understanding

Multi-task fine-tuning:

- Supervised learning on multiple tasks
- Improves generalization

Task-specific fine-tuning:

- Adapt to specific downstream task
- Often achieves best performance

Fine-tuning Strategies

Supervised Fine-tuning:

- Provide task-specific training examples
- Model learns task from examples
- Standard approach for best performance

Multi-task Fine-tuning:

- Fine-tune on multiple tasks together
- Better generalization
- Useful when tasks are related

Few-shot Learning:

- Provide few examples in prompt
- No parameter updates
- Limited by context window
- Example: "translate: cat → chat, dog → chien, bird → ?"

Zero-shot Learning:

- Describe task in natural language
- No task examples
- Relies on pre-training knowledge
- Performance varies by task

Variants and Improvements

mT5 (Multilingual T5, 2020):

- Trained on 101 languages
- Uses mC4 (multilingual C4)
- 71 TB of web text
- Cross-lingual transfer learning
- Single model for all languages

ByT5 (Byte-level T5, 2021):

- Works directly with UTF-8 bytes
- No tokenization needed

- Better for noisy text
- Good for multiple languages
- More robust to typos

UL2 (Unified Language Learner, 2022):

- Combines different denoising strategies
- X-denoising (extreme spans)
- S-denoising (short spans)
- R-denoising (prefix-LM)
- Better generalization across tasks

Flan-T5 (2022):

- Instruction-tuned T5
- Fine-tuned on 1800+ tasks
- Tasks formatted as instructions
- Significantly better zero-shot performance
- Better instruction following
- Widely used for practical applications

LongT5 (2021):

- Efficient attention for long documents
- Transient global attention
- Handles sequences up to 16,384 tokens
- Good for document summarization

Advantages of T5

- Unified framework simplifies training and deployment
- Strong transfer learning across tasks
- Flexible: handles any text-to-text task
- Excellent for both generation and understanding
- Scalable: performance improves with size
- Multi-task learning improves generalization
- Easy to add new tasks

Limitations

- Computationally expensive for large variants
- Requires careful prompt engineering
- Fixed context window (512-1024 tokens typically)
- Pre-training requires massive compute
- Autoregressive decoding can be slow

Applications

- Machine translation
 - Text summarization
 - Question answering
 - Text classification
 - Paraphrasing
 - Grammar correction
 - Data-to-text generation
 - Conversational AI
-

GPT Models

GPT: Generative Pre-trained Transformer

Core Idea: Pre-train on language modeling, then fine-tune for downstream tasks.

GPT-1 (2018)

Architecture:

- Decoder-only Transformer
- 12 layers, 768 hidden size
- 117M parameters
- Unidirectional (left-to-right)

Pre-training:

- Objective: Predict next token
- Dataset: BooksCorpus (7,000 books)
- Autoregressive language modeling

Fine-tuning:

- Task-specific input transformations
- Minimal architecture changes
- Demonstrated transfer learning effectiveness

GPT-2 (2019)**Architecture:**

- Larger decoder-only Transformer
- 4 sizes: Small (117M), Medium (345M), Large (762M), XL (1.5B)
- 48 layers, 1600 hidden size (XL)
- Layer normalization moved to input of sub-blocks

Pre-training:

- Dataset: WebText (40GB, 8M web pages)
- Higher quality dataset
- Longer training

Zero-Shot Learning:

- No fine-tuning needed for many tasks
- Task conditioning via prompts
- Demonstrated emergence of capabilities

Controversial Release:

- Initially withheld due to misuse concerns
- Staged release of models
- Sparked AI safety discussions

GPT-3 (2020)**Architecture:**

- 175 billion parameters
- 96 layers, 12,288 hidden size
- 96 attention heads
- Context window: 2048 tokens

Pre-training:

- Dataset: 570GB text (filtered Common Crawl, WebText2, Books, Wikipedia)
- 300 billion tokens
- Massive computational requirements

Few-Shot Learning:

- In-context learning without gradient updates
- Provide task examples in prompt
- No fine-tuning needed

Emergent Capabilities:

- Arithmetic, code generation
- Translation, summarization
- Creative writing, reasoning
- Common sense knowledge

API-Only:

- Not open-sourced
- Available through OpenAI API
- Commercial applications

GPT-3.5 (2022)

Improvements:

- Base model + instruction tuning
- RLHF (Reinforcement Learning from Human Feedback)
- Better instruction following
- More helpful, harmless, honest

Variants:

- text-davinci-003: Instruction-tuned
- gpt-3.5-turbo: Optimized for chat
- Code-capable versions

GPT-4 (2023)

Architecture:

- Multimodal (text and images)
- Exact size not disclosed
- Rumored mixture of experts

Capabilities:

- Significantly better reasoning
- Advanced math and coding
- Image understanding
- Longer context (8K, 32K variants)
- More factual and aligned

Safety:

- Extensive RLHF training
- Red teaming for risks
- Improved refusal behavior

Key Concepts in GPT Series

Autoregressive Generation:

- Predict one token at a time
- Condition on all previous tokens
- Left-to-right generation

In-Context Learning:

- Learn tasks from examples in prompt
- No parameter updates
- Few-shot, one-shot, zero-shot

Prompt Engineering:

- Craft prompts to elicit desired behavior
- Critical for performance
- Includes instructions, examples, context

Scaling Laws:

- Performance scales with model size, data, compute
- Predictable improvements
- Motivates larger models

Comparison: BERT vs GPT

Feature	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention	Bidirectional	Unidirectional
Pre-training	MLM + NSP	Next token prediction
Best for	Understanding	Generation
Fine-tuning	Task-specific heads	Prompt-based
Generation	Poor	Excellent
Classification	Excellent	Good with prompting

Advanced Architectures

Encoder-Decoder Models

BART (Bidirectional and Auto-Regressive Transformers, 2019):

- Combines BERT and GPT ideas
- Pre-training: Corrupt document, reconstruct it
- Multiple noise functions (masking, deletion, permutation)
- Strong for generation tasks
- Excellent for summarization

mBART (Multilingual BART):

- BART for 25+ languages
- Good cross-lingual transfer
- Machine translation

PEGASUS (2020):

- Pre-trained for abstractive summarization
- Gap sentence generation objective
- Masks entire sentences
- State-of-the-art summarization

Efficient Transformers

ALBERT (A Lite BERT):

- Factorized embeddings
- Cross-layer parameter sharing
- 89% fewer parameters than BERT-large

MobileBERT:

- Compact BERT for mobile devices
- 4.3x smaller, 5.5x faster
- Knowledge distillation

TinyBERT:

- Extremely small (14.5M parameters)
- 7.5x smaller, 9.4x faster
- Two-stage distillation

Domain-Specific Models

BioBERT:

- BERT pre-trained on biomedical text
- PubMed abstracts, PMC full-text
- Better for biomedical NER, relation extraction

SciBERT:

- BERT for scientific text
- Trained on 1.14M papers
- Custom vocabulary

ClinicalBERT:

- BERT for clinical notes
- MIMIC-III dataset
- Medical concept extraction

FinBERT:

- BERT for financial text
- Sentiment analysis for finance
- Risk classification

LegalBERT:

- BERT for legal documents
- Case law, legislation
- Legal NER and classification

Retrieval-Augmented Models

RAG (Retrieval-Augmented Generation):

- Combines retrieval with generation
- Retrieve relevant documents
- Use as additional context
- Better factuality

REALM (Retrieval-Augmented Language Model):

- Neural retrieval integrated into pre-training
- End-to-end differentiable
- Improved knowledge-intensive tasks

DPR (Dense Passage Retrieval):

- Dense embeddings for retrieval
- BERT-based encoders
- Better than sparse methods (BM25)

Vision-Language Models

CLIP (Contrastive Language-Image Pre-training):

- Joint text-image embeddings

- Zero-shot image classification
- Learned from 400M image-text pairs

ALIGN:

- Similar to CLIP
- Larger scale (1.8B pairs)
- Noisy data, simple approach

DALL-E:

- Text-to-image generation
- Autoregressive Transformer
- Creative image synthesis

Code Models

Codex (GPT for Code):

- GPT-3 fine-tuned on code
- Powers GitHub Copilot
- Multi-language code generation

CodeBERT:

- BERT for programming languages
- Pre-trained on code-comment pairs
- Code search, documentation

CodeT5:

- T5 for code understanding and generation
- Identifier-aware pre-training
- Multiple programming languages

Common NLP Tasks

1. Text Classification

Sentiment Analysis:

- Classify text by sentiment (positive, negative, neutral)

- Fine-grained (1-5 stars) or coarse (binary)
- Applications: Product reviews, social media monitoring

Topic Classification:

- Assign topics/categories to documents
- Multi-class or multi-label
- Applications: News categorization, email routing

Intent Detection:

- Identify user's intent in text
- Critical for chatbots, voice assistants
- Often combined with slot filling

Spam Detection:

- Classify messages as spam or not
- Email filtering, comment moderation

Emotion Detection:

- Classify emotions (joy, anger, sadness, fear, etc.)
- More fine-grained than sentiment

2. Sequence Labeling

Named Entity Recognition (NER):

- Identify and classify named entities
- Person, organization, location, date, etc.
- IOB tagging scheme

Part-of-Speech (POS) Tagging:

- Label each word with grammatical role
- Noun, verb, adjective, etc.

Chunking:

- Identify phrases (noun phrases, verb phrases)
- Shallow parsing

Slot Filling:

- Extract structured information
- Common in dialogue systems
- Example: Extract date, time, location from booking request

3. Sequence-to-Sequence Tasks

Machine Translation:

- Translate text between languages
- Neural MT (Transformer-based) is state-of-the-art
- Challenges: low-resource languages, domain adaptation

Text Summarization:

- **Extractive:** Select important sentences
- **Abstractive:** Generate new summary
- Single-document or multi-document

Paraphrasing:

- Rewrite text with same meaning
- Style transfer (formal ↔ casual)
- Simplification

Data-to-Text Generation:

- Generate text from structured data
- Tables, knowledge graphs, databases
- Applications: Report generation, descriptions

4. Question Answering

Extractive QA:

- Answer is span in given text
- SQuAD-style tasks
- BERT-based models excel

Generative QA:

- Generate answer from knowledge

- May not be in source text
- GPT-style models

Open-Domain QA:

- Answer questions about anything
- Requires retrieval + reading
- RAG models

Multi-Hop QA:

- Requires reasoning over multiple facts
- HotpotQA, ComplexWebQuestions
- Challenging for current models

Conversational QA:

- QA in dialogue context
- Coreference resolution needed
- Follow-up questions

5. Text Generation

Story Generation:

- Creative writing, narratives
- GPT models excel
- Coherence, creativity challenges

Dialogue Generation:

- Chatbots, virtual assistants
- Context tracking, personality
- Open-domain vs task-oriented

Code Generation:

- Generate code from natural language
- Codex, CodeT5
- Applications: Programming assistants

Poetry Generation:

- Generate poems with style, meter, rhyme
- GPT-2/3 show surprising creativity

6. Information Extraction

Relation Extraction:

- Identify relationships between entities
- "John works at Microsoft" → (John, works_at, Microsoft)
- Knowledge base construction

Event Extraction:

- Identify events and participants
- Who did what, when, where, why
- Temporal reasoning

Knowledge Graph Construction:

- Build structured knowledge from text
- Entity linking, relation extraction
- Applications: Search, QA

7. Information Retrieval

Document Ranking:

- Rank documents by relevance
- Search engines
- BM25, BERT-based rankers

Semantic Search:

- Find semantically similar content
- Dense retrieval (SBERT, DPR)
- Beyond keyword matching

Passage Retrieval:

- Find relevant passages for QA
- First stage of open-domain QA
- Dense vs sparse retrieval

8. Text Matching

Natural Language Inference (NLI):

- Determine relationship between sentences
- Entailment, contradiction, neutral
- MNLI, SNLI datasets

Semantic Similarity:

- Measure meaning similarity
- Regression or classification
- STS-B dataset

Paraphrase Detection:

- Determine if sentences have same meaning
- Binary classification
- QQP, MRPC datasets

9. Other Tasks

Text Simplification:

- Rewrite complex text to be simpler
- Preserve meaning, reduce complexity
- Applications: Accessibility, education

Grammatical Error Correction:

- Identify and correct grammar errors
- Spelling, syntax, style
- Seq2seq models

Text Style Transfer:

- Change style while preserving content
- Formal ↔ informal
- Sentiment flipping

Coreference Resolution:

- Identify when expressions refer to same entity

- Crucial for understanding
- Difficult task

Temporal Reasoning:

- Understand and reason about time
- Event ordering, duration
- Challenging for current models

Common Sense Reasoning:

- Tasks requiring world knowledge
 - COPA, HellaSwag, WinoGrande
 - Still challenging despite large models
-

Evaluation Metrics

Classification Metrics

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$

- Simple, intuitive
- Misleading for imbalanced data

Precision: $TP / (TP + FP)$

- Of predicted positives, how many correct?
- Important when false positives costly

Recall (Sensitivity): $TP / (TP + FN)$

- Of actual positives, how many found?
- Important when false negatives costly

F1 Score: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- Harmonic mean of precision and recall
- Balanced measure

F-beta Score: $(1 + \beta^2) \times (\text{Precision} \times \text{Recall}) / (\beta^2 \times \text{Precision} + \text{Recall})$

- $\beta > 1$: Favor recall

- $\beta < 1$: Favor precision

Macro F1: Average F1 across all classes

- Equal weight to each class
- Good for imbalanced datasets

Micro F1: Calculate F1 from total TP, FP, FN

- Weighted by class frequency
- Same as accuracy for multi-class

Matthews Correlation Coefficient (MCC):

- Correlation between predictions and truth
- Range: -1 to 1
- Good for imbalanced data

ROC-AUC:

- Area under ROC curve
- Measures classifier performance across thresholds
- Good for comparing models

Sequence Labeling Metrics

Token-Level Accuracy:

- Percentage of correctly labeled tokens
- May not reflect entity-level performance

Entity-Level F1:

- Precision/recall for complete entities
- Strict: All tokens must match
- Relaxed: Partial credit for overlap

Span-Level F1:

- For NER, chunking
- Counts exact span matches

Language Generation Metrics

BLEU (Bilingual Evaluation Understudy):

- Measures n-gram overlap with references
- BLEU-1, BLEU-2, ..., BLEU-4
- Range: 0-100 (higher better)
- Common for machine translation
- **Limitation:** Doesn't capture meaning, rewards literal matches

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):

- ROUGE-N: N-gram overlap
- ROUGE-L: Longest common subsequence
- ROUGE-W: Weighted LCS
- Common for summarization
- Recall-focused

METEOR:

- Considers synonyms, stemming, paraphrasing
- Better correlation with human judgment
- Precision-recall harmonic mean

CIDEr (Consensus-based Image Description Evaluation):

- TF-IDF weighted n-gram matching
- Multiple references
- Common for image captioning

SPICE:

- Semantic propositional content
- Graph-based matching
- Captures objects, attributes, relations

BERTScore:

- Uses BERT embeddings to compare texts
- Token-level semantic similarity
- Better correlation with human judgment

- More robust than n-gram metrics

BLEURT:

- Learned metric using BERT
- Trained on human ratings
- Better than BLEU for correlation

Perplexity:

- How well model predicts text
- $PPL = \exp(\text{cross-entropy loss})$
- Lower is better
- Common for language models

Question Answering Metrics

Exact Match (EM):

- 1 if predicted answer exactly matches
- 0 otherwise
- Strict, may underestimate performance

F1 Score:

- Token-level F1 between prediction and reference
- More forgiving than EM
- Partial credit for overlaps

Mean Reciprocal Rank (MRR):

- For ranking tasks
- $MRR = \text{avg}(1 / \text{rank of correct answer})$
- Higher is better

Retrieval Metrics

Precision@K:

- Precision in top K results
- $P@10$ common

Recall@K:

- Recall in top K results

Mean Average Precision (MAP):

- Average precision across queries
- Considers ranking

Normalized Discounted Cumulative Gain (NDCG):

- Considers relevance levels (not just binary)
- Position-weighted
- Gold standard for ranking

Hit Rate:

- Percentage of queries with relevant result in top K

Human Evaluation

Often necessary for:

- Generation quality
- Coherence, fluency
- Factual accuracy
- Helpfulness

Methods:

- Likert scales (1-5 ratings)
- Pairwise comparison (A vs B)
- Error analysis
- A/B testing

Metrics:

- Fluency, coherence, relevance
 - Informativeness
 - Factual correctness
 - Human preference
-

Prompt Engineering

What is Prompt Engineering?

The art and science of designing inputs to language models to elicit desired outputs.

Basic Prompting Techniques

Zero-Shot Prompting:

- No examples provided
- Just task description
- Example: "Translate to French: Hello"

Few-Shot Prompting:

- Provide examples in prompt
- Model learns pattern
- Example:

Translate to French:
Hello → Bonjour
Goodbye → Au revoir
Thank you → ?

One-Shot Prompting:

- Single example
- Middle ground

Advanced Prompting Strategies

Chain-of-Thought (CoT) Prompting:

- Include reasoning steps in examples
- "Let's think step by step"
- Dramatically improves reasoning
- Especially effective for math, logic

Self-Consistency:

- Generate multiple reasoning paths
- Take majority vote

- Improves accuracy

Tree of Thoughts:

- Explore multiple reasoning branches
- Backtrack and prune
- More thorough exploration

ReAct (Reasoning + Acting):

- Interleave reasoning and actions
- For interactive tasks
- "Thought: ..., Action: ..., Observation: ..."

Least-to-Most Prompting:

- Break problem into subproblems
- Solve sequentially
- Build on previous solutions

Prompt Design Principles

Be Specific:

- Clear, detailed instructions
- Specify format, style, length
- Example: "Write a 3-sentence summary"

Provide Context:

- Background information
- Relevant details
- Constraints

Use Delimiters:

- Separate instruction from input
- Example: ""text"", ###, ---
- Helps model parse structure

Request Structured Output:

- JSON, XML, tables

- Easier to parse
- Example: "Return as JSON with keys: sentiment, confidence"

Iterative Refinement:

- Start simple, add complexity
- Test and refine
- A/B test different phrasings

Role Prompting

Assign a role to the model:

- "You are an expert Python programmer"
- "Act as a creative writing teacher"
- Influences tone, depth, perspective

Instruction Tuning

Flan, InstructGPT approach:

- Train model on instruction-following tasks
- Better zero-shot performance
- More helpful, harmless, honest

Prompt Templates

Reusable patterns:

```
Task: {task_description}  
Input: {input_text}  
Output format: {format}  
Additional constraints: {constraints}
```

Common Pitfalls

Ambiguity:

- Vague instructions lead to poor results
- Be explicit

Information Overload:

- Too much context confuses model

- Keep prompts focused

Biased Examples:

- Examples can introduce bias
- Diverse, representative examples

Prompt Injection:

- Security risk: malicious prompts
 - Sanitize user inputs
 - Use system prompts
-

Fine-tuning Strategies

Transfer Learning Paradigm

Pre-training → Fine-tuning:

1. Pre-train on large unlabeled corpus
2. Fine-tune on task-specific labeled data
3. Achieves strong performance with less data

Full Fine-tuning

Process:

- Update all model parameters
- Requires task-specific dataset
- Generally best performance

Considerations:

- Requires significant compute
- Risk of catastrophic forgetting
- One model per task

Best practices:

- Lower learning rate than pre-training ($1e-5$ to $5e-5$)
- Few epochs (2-4 typically)
- Small batch size (8-32)

- Gradient clipping
- Warmup steps
- Early stopping

Parameter-Efficient Fine-tuning

Motivation:

- Full fine-tuning expensive
- Multiple task-specific models costly
- Share base model, swap task modules

Adapter Layers:

- Small bottleneck layers inserted in Transformer
- Only train adapters (1-4% of parameters)
- Base model frozen
- Competitive performance

LoRA (Low-Rank Adaptation):

- Decompose weight updates as low-rank matrices
- $W' = W + BA$ (B and A are low-rank)
- Train B and A only
- 0.1-1% trainable parameters
- No inference latency

Prefix Tuning:

- Prepend trainable vectors to input
- Only optimize prefix parameters
- Base model frozen
- Good for generation tasks

Prompt Tuning:

- Similar to prefix tuning
- Soft prompts (continuous vectors)
- Very few parameters

BitFit:

- Only fine-tune bias terms
- Surprisingly effective
- 0.1% of parameters

Multi-Task Learning

Joint Training:

- Train on multiple tasks simultaneously
- Shared representations
- Improved generalization
- Task interference possible

Sequential Fine-tuning:

- Fine-tune on tasks sequentially
- Intermediate task transfer
- Order matters

Curriculum Learning:

- Order tasks by difficulty
- Start easy, progress to hard
- Can improve final performance

Continual Learning

Challenge:

- Catastrophic forgetting
- Model forgets previous tasks

Strategies:

- Elastic Weight Consolidation (EWC)
- Progressive Neural Networks
- Experience Replay
- Parameter isolation

Data Augmentation

Back Translation:

- Translate to another language and back
- Generates paraphrases
- Increases diversity

Synonym Replacement:

- Replace words with synonyms
- Maintains meaning

Random Insertion/Deletion:

- Add/remove words
- Makes model more robust

Mixup:

- Interpolate examples
- Regularization effect

Contextual Word Embeddings:

- Replace words with similar contextual embeddings
- BERT-based augmentation

Regularization Techniques

Dropout:

- Randomly drop units during training
- Prevents overfitting

Weight Decay:

- L2 regularization on weights
- Prevents large weights

Label Smoothing:

- Soften target distributions
- Prevents overconfidence

Mixup/Manifold Mixup:

- Interpolate examples and labels

- Encourages smoothness

Domain Adaptation

Challenge:

- Model trained on source domain
- Deploy on target domain
- Distribution shift

Strategies:

- Continued pre-training on target domain
- Gradual unfreezing
- Domain-adversarial training
- Self-training on unlabeled target data

Few-Shot and Zero-Shot Learning

Few-Shot:

- Limited labeled examples (5-100)
- Meta-learning
- Prototypical networks
- GPT-3 style in-context learning

Zero-Shot:

- No task-specific training data
- Rely on pre-training
- Natural language task descriptions
- T5, GPT-3 approaches

NLP Challenges

Ambiguity

Lexical Ambiguity:

- Words with multiple meanings
- "bank" → financial institution vs. river bank

- Context needed to resolve

Syntactic Ambiguity:

- Multiple valid parse trees
- "I saw the man with the telescope"
- Who has the telescope?

Semantic Ambiguity:

- Unclear meaning even with syntax
- Metaphors, idioms
- "It's raining cats and dogs"

Pragmatic Ambiguity:

- Context-dependent meaning
- Sarcasm, irony
- "Great, another meeting"

Context Understanding

Long-Range Dependencies:

- Information far apart in text
- Transformers help but still limited
- Context window constraints

Coreference Resolution:

- Identifying referents
- "John went to store. He bought milk."
- Challenging across sentences

World Knowledge:

- Facts not in text
- Common sense reasoning
- Background assumptions

Multilinguality

Challenges:

- 7,000+ languages
- Most are low-resource
- Cross-lingual transfer
- Code-switching (mixing languages)

Approaches:

- Multilingual models (mBERT, XLM-R)
- Zero-shot cross-lingual transfer
- Translation-based methods

Data Challenges

Data Scarcity:

- Many domains/languages lack data
- Annotation expensive
- Transfer learning helps

Data Quality:

- Noisy web data
- Annotation errors
- Biased datasets

Data Privacy:

- Sensitive information
- GDPR, compliance
- Federated learning

Bias and Fairness

Types of Bias:

- Gender bias (associations, stereotypes)
- Racial bias
- Cultural bias
- Selection bias in data

Mitigation:

- Diverse training data
- Bias detection and measurement
- Debiasing techniques
- Fairness constraints

Explainability

Challenge:

- Neural models are black boxes
- Hard to explain predictions
- Trust, debugging, compliance

Approaches:

- Attention visualization
- Feature importance (LIME, SHAP)
- Probing tasks
- Adversarial examples

Robustness

Adversarial Examples:

- Small perturbations cause errors
- Deliberate or natural noise
- Spelling errors, typos

Out-of-Distribution:

- Data different from training
- Performance degradation
- Domain shift

Mitigation:

- Data augmentation
- Adversarial training
- Robust architectures

Evaluation

Metrics Don't Capture All:

- BLEU high doesn't mean good translation
- Human evaluation expensive
- Inter-annotator agreement

Dataset Artifacts:

- Models exploit dataset biases
- Don't generalize to real world
- Adversarial test sets needed

Computational Cost

Large Models:

- Training expensive (millions of dollars)
- Environmental impact
- Inference latency

Solutions:

- Model compression
- Distillation
- Quantization
- Efficient architectures

Common Sense and Reasoning

Challenges:

- Implicit knowledge
- Causal reasoning
- Physical reasoning
- Theory of mind

Current Limitations:

- Models lack true understanding
- Pattern matching vs. reasoning
- Brittleness on edge cases

Tools and Libraries

Python Libraries

Transformers (Hugging Face):

- Pre-trained models (BERT, GPT, T5, etc.)
- Easy fine-tuning
- Tokenizers, training, inference
- 100,000+ models on Hub
- Essential tool for modern NLP

SpaCy:

- Industrial-strength NLP
- Fast tokenization, POS, NER, parsing
- Pre-trained pipelines
- Production-ready

NLTK (Natural Language Toolkit):

- Classic NLP library
- Tokenizers, stemmers, corpora
- Educational, less efficient

Gensim:

- Topic modeling (LDA)
- Word2Vec, Doc2Vec
- Similarity queries
- Document clustering

AllenNLP:

- Research library built on PyTorch
- Pre-built models and components
- Interpretability tools

Flair:

- Simple NLP framework
- State-of-the-art models
- Easy to use
- Sequence labeling focus

FastText:

- Efficient text classification
- Word embeddings
- Subword information

Deep Learning Frameworks

PyTorch:

- Dynamic computation graphs
- Pythonic, flexible
- Preferred for research
- Strong NLP ecosystem

TensorFlow:

- Static graphs (TF 1.x), eager execution (TF 2.x)
- Production-focused
- TensorFlow Hub for models

JAX:

- NumPy + autograd + GPU/TPU
- Functional programming
- Very fast
- Growing NLP adoption

Training and Deployment

Ray:

- Distributed training
- Hyperparameter tuning
- Model serving

DeepSpeed:

- Microsoft's training optimization
- ZeRO optimizer
- Train huge models efficiently

Triton Inference Server:

- NVIDIA's serving solution
- Multi-framework support
- Optimized inference

ONNX Runtime:

- Cross-platform inference
- Model optimization
- Hardware acceleration

Datasets and Benchmarks

GLUE (General Language Understanding Evaluation):

- 9 NLU tasks
- CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, WNLI
- Leaderboard

SuperGLUE:

- Harder version of GLUE
- 8 tasks
- BoolQ, CB, COPA, MultiRC, ReCoRD, RTE, WiC, WSC

SQuAD (Stanford Question Answering Dataset):

- Extractive QA
- SQuAD 1.1: 100K questions
- SQuAD 2.0: Unanswerable questions

Natural Questions:

- Google search queries
- Wikipedia passages
- Short and long answers

CommonGen, CommonsenseQA:

- Common sense reasoning
- Challenging for models

Annotation Tools

Label Studio:

- Multi-type annotation
- NER, classification, etc.
- ML-assisted labeling

Prodigy:

- Active learning
- Efficient annotation
- SpaCy integration

Doccano:

- Open-source
- Text classification, sequence labeling
- Collaborative

Model Hubs

Hugging Face Hub:

- 100,000+ models
- Datasets, spaces (demos)
- Community-driven

TensorFlow Hub:

- Google's model repository
- Pre-trained models
- Easy integration

Papers With Code:

- Research + implementations
- Benchmarks, leaderboards

- Reproducibility

Quick Reference: Model Comparison

Model	Type	Best For	Params	Context	Released
BERT-Base	Encoder	Understanding	110M	512	2018
RoBERTa	Encoder	Understanding	125M	512	2019
GPT-2	Decoder	Generation	1.5B	1024	2019
GPT-3	Decoder	Few-shot	175B	2048	2020
T5-Base	Enc-Dec	Versatile	220M	512	2019
BART	Enc-Dec	Summarization	406M	1024	2019
XLNet	AR	Understanding	340M	512	2019
ELECTRA	Encoder	Efficient	335M	512	2020
DeBERTa	Encoder	SOTA	1.5B	512	2020

Learning Resources and Best Practices

Recommended Learning Path

Beginner:

1. Start with fundamentals (tokenization, embeddings)
2. Implement simple models (Naive Bayes, Logistic Regression)
3. Learn PyTorch/TensorFlow basics
4. Use pre-trained models (BERT for classification)
5. Practice on Kaggle competitions

Intermediate:

1. Deep dive into Transformers
2. Fine-tune models for specific tasks
3. Understand attention mechanisms
4. Experiment with different architectures

5. Read seminal papers

Advanced:

1. Implement models from scratch
2. Research novel architectures
3. Optimize for production
4. Contribute to open source
5. Stay current with latest research

Key Papers to Read

Foundational:

- "Attention Is All You Need" (Transformer, 2017)
- "BERT: Pre-training of Deep Bidirectional Transformers" (2018)
- "Language Models are Unsupervised Multitask Learners" (GPT-2, 2019)
- "Exploring the Limits of Transfer Learning with T5" (2019)
- "Language Models are Few-Shot Learners" (GPT-3, 2020)

Word Embeddings:

- "Efficient Estimation of Word Representations" (Word2Vec, 2013)
- "GloVe: Global Vectors for Word Representation" (2014)
- "Enriching Word Vectors with Subword Information" (FastText, 2016)

Important Variants:

- "RoBERTa: A Robustly Optimized BERT Pretraining Approach" (2019)
- "ALBERT: A Lite BERT for Self-supervised Learning" (2019)
- "DeBERTa: Decoding-enhanced BERT with Disentangled Attention" (2020)
- "ELECTRA: Pre-training Text Encoders as Discriminators" (2020)

Best Practices

Data Preparation:

- Clean and normalize text appropriately
- Use proper train/val/test splits
- Balance datasets when needed
- Augment data strategically

- Version control datasets

Model Selection:

- Start with pre-trained models
- Choose architecture based on task
- Consider computational constraints
- Benchmark multiple approaches
- Document decisions

Training:

- Monitor training/validation loss
- Use early stopping
- Save checkpoints frequently
- Track experiments (Weights & Biases, MLflow)
- Use appropriate learning rate schedules

Evaluation:

- Use multiple metrics
- Error analysis is crucial
- Test on out-of-distribution data
- Human evaluation when possible
- A/B testing in production

Deployment:

- Optimize for latency (quantization, distillation)
- Monitor model performance
- Version models properly
- Plan for updates
- Handle edge cases gracefully

Common Mistakes to Avoid

1. **Data Leakage:** Test data in training
2. **Ignoring Class Imbalance:** Leads to poor minority class performance
3. **Over-reliance on Accuracy:** Use appropriate metrics
4. **Not Checking for Bias:** Can perpetuate harmful stereotypes

5. **Insufficient Error Analysis:** Misses improvement opportunities
 6. **Overfitting to Test Set:** Hyperparameter tuning on test data
 7. **Ignoring Computational Costs:** Unsustainable in production
 8. **Poor Documentation:** Makes reproduction impossible
 9. **Not Validating Assumptions:** Context window limits, tokenization issues
 10. **Neglecting Edge Cases:** Real-world data is messy
-

Future Directions in NLP

Emerging Trends

Multimodal Models:

- Vision + Language (CLIP, Flamingo)
- Audio + Text
- Unified representations
- Better grounding of language

Efficient Models:

- Smaller models with comparable performance
- Distillation, pruning, quantization
- Edge deployment
- Environmental sustainability

Retrieval-Augmented Generation:

- Combining retrieval with generation
- Better factuality
- Access to external knowledge
- More controllable outputs

Instruction Tuning:

- Better instruction following
- Natural language interfaces
- Few-shot capabilities
- Alignment with human intent

Long-Context Models:

- Handling thousands of tokens
- Document-level understanding
- Better memory and recall
- Efficient attention mechanisms

Constitutional AI:

- Self-improvement through feedback
- Harmlessness, helpfulness, honesty
- Reduced human supervision
- More aligned systems

Open Problems

True Understanding:

- Models lack genuine comprehension
- Statistical patterns vs. meaning
- Grounding in real world
- Causality and reasoning

Data Efficiency:

- Humans learn from much less data
- Few-shot learning improvements
- Meta-learning advances
- Self-supervised innovations

Robustness:

- Adversarial examples
- Out-of-distribution generalization
- Systematic generalization
- Compositional understanding

Interpretability:

- Understanding model decisions
- Debugging failures
- Building trust

- Meeting regulations

Ethical AI:

- Mitigating bias
- Privacy preservation
- Fairness guarantees
- Responsible deployment

Multilingual Equity:

- Low-resource languages
 - Cross-lingual transfer
 - Cultural sensitivity
 - Linguistic diversity
-

Glossary of Terms

Attention: Mechanism allowing models to focus on relevant parts of input

Autoencoding: Reconstruction-based pre-training (like BERT's MLM)

Autoregressive: Predicting next token given previous ones (like GPT)

BLEU: Metric for evaluating generation based on n-gram overlap

Contextualized Embeddings: Word representations that change based on context

Curriculum Learning: Training on progressively harder examples

Distillation: Transferring knowledge from large model to small one

Encoder-Decoder: Architecture with separate encoding and decoding components

Few-Shot Learning: Learning from few examples

Fine-tuning: Adapting pre-trained model to specific task

Hallucination: Model generating plausible but false information

In-Context Learning: Learning from examples in prompt without parameter updates

Knowledge Distillation: Training small model to mimic large one

LoRA: Low-rank adaptation for efficient fine-tuning

Masked Language Modeling: Predicting masked tokens (BERT's pre-training)

Multi-Head Attention: Multiple attention mechanisms in parallel

Perplexity: Measure of how well model predicts text (lower is better)

Positional Encoding: Adding position information to embeddings

Pre-training: Initial training on large corpus before fine-tuning

Prompt Engineering: Designing inputs to elicit desired model behavior

RAG: Retrieval-Augmented Generation

RLHF: Reinforcement Learning from Human Feedback

Self-Attention: Attention over same sequence

Sequence-to-Sequence: Mapping input sequence to output sequence

Softmax: Function converting scores to probability distribution

Subword Tokenization: Breaking words into smaller units

Temperature: Controls randomness in generation (higher = more random)

Transfer Learning: Using knowledge from one task for another

Transformer: Architecture based on self-attention

Zero-Shot Learning: Performing task without task-specific training

Key Takeaways

1. **Transformers revolutionized NLP** through self-attention and parallel processing, enabling better modeling of long-range dependencies
2. **Pre-training + fine-tuning** is the dominant paradigm - learn general language understanding, then specialize for tasks
3. **BERT excels at understanding** through deep bidirectional context and masked language modeling
4. **GPT excels at generation** through autoregressive modeling and massive scale
5. **T5 unifies all tasks** into text-to-text format for maximum flexibility and simplicity
6. **Contextual embeddings** capture word meaning based on context, far superior to static embeddings
7. **Attention mechanisms** allow models to dynamically focus on relevant information
8. **Transfer learning** enables solving new tasks with limited labeled data
9. **Prompt engineering** is crucial for getting best performance from large language models
10. **Scaling laws** show that larger models, more data, and more compute lead to better performance
11. **Evaluation is multifaceted** - no single metric captures all aspects of performance
12. **Practical considerations** matter: efficiency, robustness, fairness, and interpretability

13. **NLP is rapidly evolving** - stay current with research, but fundamentals remain important
 14. **Understanding architectures** helps choose the right tool for each task
 15. **Ethical considerations** are paramount - bias, privacy, environmental impact, and responsible deployment
-

Quick Command Reference

Using Hugging Face Transformers

```
python
```

```
# Load pre-trained model
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

# Tokenize text
inputs = tokenizer("Hello world", return_tensors="pt")

# Get embeddings
outputs = model(**inputs)
embeddings = outputs.last_hidden_state

# Text classification
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
result = classifier("I love NLP!")

# Question answering
qa = pipeline("question-answering")
result = qa(question="What is NLP?", context="NLP is...")

# Text generation
generator = pipeline("text-generation", model="gpt2")
result = generator("Once upon a time", max_length=50)

# Named Entity Recognition
ner = pipeline("ner")
result = ner("John works at Microsoft in Seattle")

# Translation
translator = pipeline("translation_en_to_fr")
result = translator("Hello, how are you?")

# Summarization
summarizer = pipeline("summarization")
result = summarizer(long_article, max_length=130)
```

Fine-tuning Example

```
python
```

```
from transformers import AutoModelForSequenceClassification, Trainer, TrainingArguments
```

```
# Load model
```

```
model = AutoModelForSequenceClassification.from_pretrained(  
    "bert-base-uncased", num_labels=2  
)
```

```
# Define training arguments
```

```
training_args = TrainingArguments(  
    output_dir="./results",  
    num_train_epochs=3,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=64,  
    warmup_steps=500,  
    weight_decay=0.01,  
    logging_dir="./logs",  
    evaluation_strategy="epoch",  
)
```

```
# Create trainer
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=eval_dataset,  
)
```

```
# Train
```

```
trainer.train()
```

```
# Evaluate
```

```
results = trainer.evaluate()
```

SpaCy Example

```
python
```

```
import spacy

# Load model
nlp = spacy.load("en_core_web_sm")

# Process text
doc = nlp("Apple is looking at buying U.K. startup for $1 billion")

# Tokenization
tokens = [token.text for token in doc]

# POS tagging
pos = [(token.text, token.pos_) for token in doc]

# Named Entity Recognition
entities = [(ent.text, ent.label_) for ent in doc.ents]

# Dependency parsing
deps = [(token.text, token.dep_, token.head.text) for token in doc]

# Sentence segmentation
sentences = [sent.text for sent in doc.sents]
```

This comprehensive cheat sheet covers the fundamentals through advanced topics in NLP, providing a solid foundation for both learning and practical application. Keep it handy as a reference guide!